

Laboratorio 6 - Uso di mesh e calcolo dell'illuminazione

In questa esercitazione ripartiamo dal programma del laboratorio precedente, estendendolo con le seguenti caratteristiche:

- Caricamento di un oggetto rappresentato da una mesh e creazione della scena mediante tale oggetto, invece che tramite triangoli definiti dal programma;
- Rendering dell'oggetto con un colore (materiale) uniforme, calcolando l'equazione dell'illuminazione con tre modalità diverse: per vertice con interpolazione del colore (Gouraud); per vertice con interpolazione della normale (Phong); per faccia (flat).

Capitolo 0 - Template: caricamento della mesh

Il programma fornito legge una mesh da file: il nome del file contenente la mesh viene specificato come parametro del programma da linea di comando. La mesh viene:

1. caricata in una struttura dati interna indicizzata;
2. riscalata al cubo unitario $[-1,1] \times [-1,1] \times [-1,1]$
3. trasferita alla GPU, sempre in formato indicizzato. Per il momento, assegniamo lo stesso colore a tutti i vertici.

A questo scopo, il programma introduce una nuova classe **Mesh** che contiene tutte le informazioni relative alla mesh e le funzioni per leggerla da file, normalizzarla e fornirne una versione adatta al trasferimento alla GPU.

Queste operazioni avvengono nel contesto della classe **Scene**, modificando il codice che in precedenza definiva direttamente l'oggetto (due triangoli) e lo trasferiva alla GPU.

Per il resto, il programma esegue le stesse funzionalità precedenti. Facendo il rendering della mesh, ne vediamo la sagoma colorata di un colore uniforme. Otteniamo questo risultato perché tutti i frammenti generati dai triangoli che compongono la mesh ricevono lo stesso colore.

Note:

- il CMakeLists.txt ora scarica anche un file di dati, che vengono posti nella cartella `/build/data`
- le funzionalità di zoom e dolly sono state leggermente modificate per avere un effetto più uniforme (invece di incrementare le distanze per un termine costante, vengono modificate per una percentuale del loro valore corrente)
- per evidenziare le variazioni di prospettiva a seconda dell'obiettivo usato, sono state aggiunte con la pressione dei tasti 'n', 't' e 'w' le funzionalità di cambio dell'inquadratura da normale a teleobiettivo a grandangolo: tali funzionalità cambiano contemporaneamente la distanza focale e la distanza

dell'oggetto e dei piani ci clip, in modo da visualizzare l'oggetto sempre a tutto campo, ma inquadrato con obiettivi diversi.

Capitolo 1 - Calcolo delle normali

Per il calcolo dell'illuminazione, la direzione normale alla superficie deve essere nota per ogni punto in cui il calcolo viene effettuato. Nel seguito, consideriamo tre tipi diversi di shading: nel Gouraud shading, l'illuminazione viene calcolata per vertice; nel Phong shading e nel flat shading, l'illuminazione viene calcolata per frammento.

Il nostro obiettivo è stimare una normale per ogni vertice, calcolata come se la mesh rappresentasse un'approssimazione poliedrale di un oggetto liscio: la normale di un vertice viene calcolata come somma pesata delle normali ai triangoli incidenti su di esso.

Dato un triangolo $t = (v_0, v_1, v_2)$, i cui vertici siano elencati in senso antiorario quando t viene visto dall'esterno della mesh (front face), il prodotto vettore $N_t = (v_1 - v_0) \times (v_2 - v_0)$ fornisce un vettore che è:

- perpendicolare a t ;
- diretto verso l'esterno della mesh;
- di modulo pari al doppio dell'area di t .

La normale di un vertice v viene stimata con la formula $\frac{\sum_{i=1}^k N_{t_i}}{\|\sum_{i=1}^k N_{t_i}\|}$, che corrisponde alla media delle normali dei k triangoli t_i incidenti su v , ciascuna pesata dall'area del corrispondente triangolo.

Per effettuare questo calcolo procediamo nel modo seguente:

1. allochiamo all'interno della classe **Mesh** un vector **normals** di **vec3**, analogo a quello che contiene le posizioni dei vertici, dimensionato allo stesso numero di elementi e inizializzato con tutti zeri (vettori nulli); useremo questo vector per immagazzinare le normali;
2. creiamo una funzione privata **Mesh::compute_normals**, che verrà chiamata dal costruttore di **Mesh**.
3. In questa funzione, cicliamo su tutti i triangoli della mesh; per ogni triangolo t , calcoliamo la sua normale "pesata" N_t come sopra e accumuliamo tale vettore sommandolo alle tre entry di **normals** che corrispondono alle posizioni dei tre vertici di t : notiamo che i tre interi che costituiscono l'informazione associata a t nella lista dei triangoli servono sia per accedere alle coordinate dei suoi vertici, sia per accumulare il vettore calcolato nelle entry corrispondenti agli stessi vertici nel vector **normals**. Alla fine del ciclo, tutti i vertici hanno ricevuto i contributi da tutti i triangoli incidenti, la cui somma costituisce il numeratore della formula sopra.
4. Infine, facciamo un ciclo sul vector **normals** e applichiamo la funzione **glm::normalize** a ogni entry.

Notiamo che il colore per vertice usato nel Capitolo 0 ha poco senso, dato che tutti i vertici ricevono lo stesso colore; anche i capitoli successivi non avranno bisogno di un “colore” per vertice, perché i colori saranno ottenuti dal calcolo di illuminazione e shading. La maggior parte delle informazioni necessarie per il calcolo sono uguali per tutti i vertici, quindi possono essere passate mediante uniform. Tuttavia, la normale deve essere indicata per vertice (almeno per i metodi di Gouraud e di Phong).

Sostituiamo quindi al buffer dei colori un analogo buffer di normali (costituito comunque di tre float per vertice). La struttura del buffer GPU resta invariata, ma il contenuto assegnato al secondo attributo cambia. Per coerenza, cambiamo anche i nomi, sia nel programma sia nello shader.

Se non facciamo altre modifiche, facendo girare il programma vedremo il nostro oggetto con una colorazione artificiale, che ora è diversa da punto a punto e piuttosto scura. Cosa è successo? Le normali vengono interpolate dal rasterizzatore e interpretate come colori dal fragment shader: ad ogni normale viene quindi associato un *falso colore* che varia uniformemente sulla superficie dell’oggetto al variare della direzione interpolata della normale.

Per schiarire un po’ il colore possiamo intervenire sul fragment shader sostituendo la formula di calcolo del colore come segue:

```
fragment_color = vec4 (interpolated_normal * 0.5 + 0.5, 1.0);
```

Esistono formule più accurate che usano la direzione normale per calcolare le coordinate cilindriche o sferiche (Gauss map) e queste ultime per calcolare un colore in formato HSV o HSL, che viene infine trasformato in RGB. Il linguaggio glsl permette di fare queste operazioni in poche linee di codice. I falsi colori delle normali sono molto utili per fare debugging nei programmi di shading.

Capitolo 2 - Gouraud shading

Nello shading di Gouraud, l’equazione dell’illuminazione viene calcolata dal vertex shader e il colore ottenuto viene associato al vertice e passato come parametro al rasterizzatore per essere interpolato con quelli degli altri vertici della primitiva.

Il fragment shader riceve direttamente il colore interpolato e non deve far altro che passarlo oltre sulla pipeline. Quindi, ripristiniamo il fragment shader allo stesso codice usato in precedenza (prima della modifica con le normali).

Il grosso del lavoro sull’illuminazione viene fatto dal vertex shader. Tuttavia, prima di modificarlo, dobbiamo preparare i dati necessari, che manterremo nella classe **CameraLights**, che è un’evoluzione della precedente classe **Camera** che incorpora anche tutte le nuove informazioni.

Per poter calcolare le equazioni di illuminazione, sono necessari i seguenti ingredienti:

- Parametri geometrici (diversi per ogni punto):

- Posizione e normale del punto da illuminare (ce li abbiamo)
- Direzione da cui proviene la luce che illumina il punto
- Direzione di vista rispetto al punto stesso
- Parametri di materiale (oggetto uniforme: gli stessi per tutti i punti):
 - Coefficiente di riflessione diffusa
 - Coefficiente di riflessione speculare e esponente di shininess
 - Coefficiente di riflessione ambiente
- Parametri delle luci (gli stessi per tutti i punti):
 - Colore della luce diretta
 - Colore della luce ambiente

Le direzioni di provenienza della luce e di osservazione sono desumibili dalle posizioni relative del punto, della luce e dell'osservatore. È necessario che queste posizioni siano espresse nello stesso sistema di riferimento.

In sintesi, a parte posizioni e normali che saranno attribuiti in input allo shader, avremo bisogno di passare le seguenti uniform, che vanno definite nella `CameraLights`:

- posizione della luce in WC
- posizione dell'osservatore in WC: se V è la matrice di vista (che abbiamo già), la posizione dell'osservatore si ottiene applicando la matrice V^{-1} all'origine
- parametri di materiale e delle luci: in tutto sono 6 float
- (volendo illuminare la scena con più di una luce, bisognerà ripetere posizione e intensità per ogni luce; le luci si possono immagazzinare in un array e bisogna che il numero di luci nel programma sia coerente con quello atteso dallo shader).

Nota: Può essere utile usare una luce solidale con la camera, intendendo che le sue coordinate siano espresse nel sistema di vista anziché in quello del mondo. Nel nostro caso, siccome la camera orbita attorno alla scena e la luce la segue, affinché il calcolo risulti corretto, sarà necessario moltiplicare anche la posizione della camera per la matrice V^{-1} , in modo da ottenere le sue coordinate in WC.

Il vertex shader, oltre a gestire le trasformazioni come in precedenza:

1. riceve come attributi la posizione e la normale al punto e come uniform tutti gli altri parametri;
2. calcola la direzione della luce e quella dell'osservatore per differenza tra le posizioni degli stessi e la posizione del punto;
3. calcola l'equazione dell'illuminazione (vedi slide di teoria) e ottiene il colore da assegnare al punto.

Nel caso di più sorgenti di luce: l'equazione va calcolata per ogni sorgente e i contributi delle diverse luci vanno sommati. Ricordarsi di fare clamp a 1 dei valori delle componenti RGB ottenute, perché non possono eccedere tale valore.

Il programma ora deve visualizzare la mesh come se fosse “liscia”. Se la risoluzione della mesh è alta e non contiene spigoli molto vivi, l'illusione di aver visualizzato

una superficie curva e liscia dovrebbe essere abbastanza convincente. Tuttavia, se la risoluzione è bassa e/o la geometria contiene spigoli vivi e vertici “puntuti”, potremo notare molti artefatti.

Capitolo 3 - Phong shading

Nello shading di Phong, l'equazione dell'illuminazione viene calcolata dal fragment shader, che riceve tutte le uniform necessarie al calcolo, precedentemente passate al vertex shader.

Il fragment shader effettua il calcolo ancora in coordinate del mondo, perché le luci e l'osservatore sono espressi in tale sistema di riferimento. Per procedere, ha bisogno che anche la posizione del punto e la sua normale siano espressi nello stesso sistema di riferimento. Questi dati però sono valori interpolati, ricevuti dal rasterizzatore, quindi devono essere prodotti per ogni vertice dal vertex shader.

Aggiungiamo quindi due attributi in output al vertex shader (eliminando invece il colore): la posizione del punto e la normale, entrambe espresse nel sistema WC. Entrambe coincidono con gli attributi ricevuti in input. Il vertex shader *oltre* a fornire la posizione del vertice trasformata tramite la matrice di vista e proiezione - come in precedenza - passa al rasterizzatore *anche* questi due nuovi attributi. Il punto prosegue quindi nella pipeline con due triple diverse di coordinate: quelle di clip ottenute dalla trasformazione di vista e proiezione, che saranno utilizzate per determinarne la geometria; e quelle originali del mondo, che saranno usate per il calcolo dell'illuminazione; a queste si aggiungono le coordinate della normale (in WC).

I corrispondenti attributi di input al fragment shader (valori interpolati) forniscono i dati necessari al calcolo dell'equazione dell'illuminazione (la stessa di prima) che decide il colore del frammento. Notiamo che la differenza sostanziale del metodo di Phong rispetto a quello di Gouraud è che invece di interpolare il colore già calcolato, interpola il valore della normale e il calcolo dell'illuminazione viene differito. Il risultato del programma dovrebbe migliorare la qualità percepita (la differenza dipende molto da quale oggetto si considera).

Capitolo 4 - Flat shading

Il flat shading si usa per geometrie poliedrali che si intendano tali e non approssimazioni di geometrie curve e lisce. L'idea è che le facce piate appaiano piate e gli spigoli appaiano tutti vivi. Ad esempio, se volessimo visualizzare un cubo o una piramide, sarebbe sbagliato utilizzare Gouraud o Phong: lo shading risultante risulterebbe artificioso.

Per realizzare il flat shading, è necessario che tutti i frammenti di un triangolo ricevano lo stesso colore. Questo requisito contrasta con la struttura della pipeline di rendering, infatti:

- non è possibile associare attributi all'element array buffer, ovvero, non possiamo specificare attributi “per triangolo”
- nella struttura usata fin qui, ogni vertice ha una sua normale, che in generale differisce dalla normale di tutte le facce incidenti su di esso (tranne il caso particolare in cui siano tutte coplanari).

Ci troviamo quindi nella situazione in cui le normali specificate ai vertici non ci forniscono informazione, mentre ci viene impedito di fornire tale informazione per triangolo. Come possiamo fare?

Una soluzione banale consiste nell'esplosione della mesh, trasformando il formato indicizzato in un formato a polygon soup (quindi solo un vertex buffer object, che contiene triple di vertici consecutivi, dove ciascuna tripla definisce un triangolo). In questo caso, i vertici vengono duplicati su tutti i loro triangoli incidenti e si può associare a ogni vertice la normale dell'unico triangolo di cui fa parte: i tre vertici dello stesso triangolo ricevono la stessa normale (quella del triangolo stesso) e tutto funziona anche usando Gouraud o Phong shading. Ma con questa struttura possiamo fare *solo* flat shading. Ci troveremmo quindi nella situazione di usare due strutture dati diverse, da caricare entrambe in GPU e usare in alternativa, secondo il tipo di shading desiderato. Questa soluzione, oltre a sprecare memoria, è poco pratica e anche pericolosa per la necessità di tenere le strutture della mesh allineate.

Per fortuna, è possibile rimediare utilizzando esattamente lo stesso programma del Phong shading e cambiando poche cose nel fragment shader. La soluzione sfrutta la possibilità del fragment shader di calcolare le derivate in spazio schermo: queste consentono con un semplice calcolo di ottenere la normale del triangolo di cui il frammento fa parte. Questa tecnica è alla base di numerose tecniche discrete di computer grafica avanzata. È semplicissima da applicare, meno da capire.

Come nel Phong shading, il fragment shader riceve come parametro in input la posizione `vWorldPos` del frammento stesso nel sistema WC. Le funzioni `glSL dFdx(vWorldPos)`, `dFdy(vWorldPos)` forniscono le derivate in spazio schermo di questo attributo. In particolare: `dFdx(vWorldPos)` dice di quanto varia il valore di `vWorldPos` se ci si sposta sul frammento adiacente in orizzontale (colonna successiva o precedente); analogamente, `dFdy(vWorldPos)` dice di quanto varia il valore di `vWorldPos` se ci si sposta sul frammento adiacente in verticale (riga successiva o precedente). Il valore normalizzato del vettore (`dFdx(vWorldPos)`, `dFdy(vWorldPos)`) pertanto fornisce la direzione normale della faccia che contiene il frammento.

Il resto del calcolo è identico a quello del Phong shading, utilizzando questa normale invece di quella interpolata, che può essere pertanto eliminata sia dal vertex shader che dal fragment shader, per evitare lavoro inutile.

Il programma deve ora visualizzare la mesh mettendo in evidenza le sue facce piatte e i suoi spigoli vivi.

Capitolo 5 - Controllo dello shading

Le differenze fra i capitoli 1, 2, 3 e 4 sono quasi tutte negli shader. Il programma principale mano a mano aggiunge nuove uniform, o ne cambia la destinazione d'uso: ad esempio alcune uniform vengono lette prima dal vertex shader, poi nei capitoli successivi vengono invece lette dal fragment shader, ma per il programma principale questo è del tutto trasparente, perché si limita a passare attributi e uniform alla GPU. È poi lo shader program che decide come vanno trattati attributi e uniform.

Modifichiamo quindi il programma per fare in modo che l'utente possa decidere interattivamente che modalità di shading usare: nella callback `handle` della keyboard, se vengono premuti rispettivamente i tasti 'f', 'g', 'p', o 'c', carichiamo e attiviamo lo shader program flat, Gouraud, Phong o campo colore delle normali.

Esercizio facoltativi:

- Lo shader che applica un campo colore falsato che rappresenta le normali usa molte meno uniform. Questo non è un grosso problema, semplicemente le `getUniform` per nomi che non esistono ritorna un handle non valido e le chiamate a `glUniform` successivo falliscono. Un codice pulito dovrebbe comunque tenere traccia di quale shader sia attivo ed evitare queste chiamate se inutili.
- per maggior efficienza, gli shader potrebbero essere caricati una volta sola all'inizio. A quel punto la `handle` può limitarsi ad attivare quello desiderato. Se si volesse aggiungere di nuovo una funzionalità di hot loading, questa potrebbe o ricaricare tutti gli shader, o aggiornare solo quello attualmente in uso.